CINEBOOK -MOVIE TICKET BOOKING SYSTEM

Lovely Professional University

DEPT.OF

COMPUTER SCIENCE & ENGINEERING

Submitted By:

NAME: KOTA VENKATESH

REGISTRATION NO: 12417035

ABSTRACT

Modern Modern e-commerce platforms have redefined customer interaction across retail, banking, and entertainment sectors. In the entertainment industry, online movie ticketing portals such as BookMyShow and District process millions of user requests every day, requiring high availability, rapid response times, and seamless user experiences. Despite these advancements, commercial reservation platforms continue to face several technical challenges, including synchronization conflicts during peak booking hours, inefficient seat allocation strategies, sub-optimal location-based recommendation systems, and limited transparency in demonstrating the underlying computational processes. These limitations can result in booking failures, inconsistent user experiences, and increased system overhead.

This project presents **CineBook**, a premium, high-performance movie ticket booking system engineered to address these challenges through the integration of fundamental **Data Structures and Algorithms (DSA)** directly into the application's business logic. Unlike conventional booking platforms that primarily rely on database queries and framework-level optimizations, CineBook incorporates algorithm-driven decision-making at every critical stage of the booking workflow. The system combines efficient computational techniques with a modern, production-inspired user interface to deliver a scalable, responsive, and educational movie reservation platform.

The application architecture follows a modular design that separates the commercial booking workflow from the educational visualization environment. During the customer booking process, **Graph-based algorithms** such as **Dijkstra's Algorithm** and **A*** Search compute the shortest travel paths between the user's location and nearby theatres by traversing weighted graph nodes representing geographical coordinates. These algorithms enable intelligent theatre recommendations based on minimum travel distance and estimated route efficiency.

Once a theatre is selected, **Binary Search** is employed to efficiently identify available show timings from chronologically ordered schedules, significantly reducing search complexity compared to traditional sequential approaches. The seat reservation subsystem combines multiple algorithmic techniques to maximize customer satisfaction. **Greedy Algorithms** prioritize the allocation of premium center seats to provide optimal viewing experiences, while **Backtracking Algorithms** recursively evaluate alternative seating arrangements to ensure adjacent seats are allocated for families and groups whenever possible. This hybrid strategy improves seat utilization while maintaining customer preferences.

To guarantee data consistency under simultaneous booking requests, CineBook integrates **Mutex Locks** and **Semaphore synchronization mechanisms** within the Node.js backend. These concurrency control techniques prevent race conditions and eliminate duplicate seat reservations by ensuring that only authorized execution threads modify seat availability during active transactions. This design closely resembles synchronization mechanisms used in real-world distributed reservation systems and demonstrates the practical application of operating system concepts within web applications.

Beyond the transactional workflow, CineBook features a dedicated **DSA Dashboard** designed as an interactive educational environment. This module provides algorithm visualizations, execution playback controls, live variable inspection panels, complexity analysis reports, traversal animations, and step-by-step execution traces. Students and developers can observe how each algorithm processes data structures internally, making the platform both a functional booking application and a practical learning tool for understanding algorithm behavior in real-world scenarios.

The system is implemented using **React**, **TypeScript**, **Node.js**, **Express.js**, and modern frontend technologies to deliver a responsive, scalable, and visually engaging interface. The modular architecture ensures maintainability, encourages code reusability, and facilitates future enhancements such as AI-driven recommendation systems, cloud deployment, microservice integration, secure authentication, payment gateway support, and real-time analytics. Performance-oriented design principles combined with optimized algorithms contribute to reduced computational complexity, faster response times, improved resource utilization, and enhanced scalability under increasing user loads.

Experimental evaluation demonstrates that integrating appropriate data structures and algorithms directly into commercial application workflows improves execution efficiency, minimizes booking conflicts, strengthens thread safety, and provides a smoother user experience. Furthermore, the inclusion of an independent visualization framework bridges the gap between theoretical computer science concepts and practical software engineering, allowing learners to understand not only how algorithms work but also why they are selected for specific business problems.

In conclusion, CineBook illustrates how the strategic integration of classical Data Structures and Algorithms within modern web architectures can simultaneously satisfy academic objectives and industry requirements. By combining efficient algorithmic processing, concurrency management, premium user interface design, and educational visualization tools, the project demonstrates a comprehensive approach to building intelligent, scalable, and reliable movie ticket booking systems that serve both end users and aspiring software engineers.

1. INTRODUCTION

1.1 Overview

The digital revolution has transformed traditional brick-and-mortar box offices into sophisticated, high-speed online ticket booking systems that enable customers to reserve movie tickets anytime and from anywhere. These platforms have significantly improved convenience by providing instant access to movie schedules, theatre information, seat availability, pricing, and secure digital transactions. As the entertainment industry continues to embrace digital transformation, online booking applications must process thousands of simultaneous requests while maintaining high performance, scalability, and reliability. Modern ticket booking systems are expected to deliver real-time seat updates, personalized recommendations, accurate theatre information, and seamless user experiences across multiple devices.

Online ticket booking systems manage several critical operations, including movie scheduling, seating availability, theatre location mapping, booking verification, and payment processing. Each of these operations requires efficient computational techniques to ensure fast response times and optimal resource utilization. With millions of transactions processed daily, these platforms demand robust software architectures capable of minimizing response latency, maximizing computational throughput, and maintaining strict data consistency during concurrent booking operations. Without efficient algorithms and synchronization mechanisms, users may experience slow searches, inefficient seat allocation, duplicate reservations, or inconsistent booking records, especially during peak hours when thousands of customers access the system simultaneously.

CineBook is designed to combine the polished user interface of a premium commercial booking platform with an educational environment that demonstrates the practical implementation of Data Structures and Algorithms (DSA). Unlike conventional booking systems where algorithmic operations remain hidden within backend services, CineBook exposes these computational processes through an interactive DSA Dashboard while preserving a clean and intuitive customer booking experience. This dual-purpose architecture enables users to understand how algorithms solve real-world business problems without affecting the usability of the booking workflow.

The system integrates several classical algorithms into different stages of the reservation pipeline. Graph algorithms such as **Dijkstra's Algorithm** and **A*** Search are utilized to identify the nearest theatres by computing the shortest travel paths between geographical locations. **Binary Search** accelerates the retrieval of available show timings from sorted schedules, reducing search complexity and improving responsiveness. **Greedy Algorithms** allocate the most desirable seats based on predefined viewing criteria, while **Backtracking Algorithms** search for adjacent seating arrangements suitable for families and groups. To ensure data integrity during simultaneous booking attempts, **Mutex Locks** and **Semaphore synchronization techniques**

coordinate concurrent transactions and eliminate race conditions that could otherwise result in duplicate seat reservations.

Beyond the commercial booking workflow, CineBook provides an interactive visualization environment where users can observe algorithm execution through animated graph traversals, step-by-step execution controls, complexity analysis, live variable inspection, and execution trace logs. This educational module bridges the gap between theoretical computer science concepts and practical software engineering by allowing learners to visualize how data structures and algorithms operate within a production-inspired application.

Developed using modern web technologies including **React**, **TypeScript**, **Node.js**, and **Express.js**, CineBook demonstrates how efficient algorithms, optimized data structures, and scalable software architecture can be integrated to build intelligent, reliable, and high-performance web applications. The project highlights that embedding Data Structures and Algorithms directly into commercial application logic not only improves computational efficiency and system reliability but also provides valuable educational insights into solving real-world engineering problems.

1.2 Problem Statement

Despite the popularity of existing booking platforms, several algorithmic and operational inefficiencies remain unresolved. These challenges form the core motivations for the development of CineBook:

1. Distance Inefficiencies: Generic platforms sort cinema listings alphabetically or based on simple region selectors, ignoring travel coordinates. Finding the absolute nearest theater requires external mapping applications.
2. Seat Recommendation Limitations: Automatic seating recommendation filters often allocate isolated seats, ignoring user preferences for middle screen views or group bookings (adjacent seats).
3. Concurrency Failures & Double Bookings: During high-demand releases, multiple booking threads can read the same seat layout concurrently, leading to race conditions where two threads reserve the same seat simultaneously.
4. Educational Gaps: Standard dashboards explain theoretical algorithms using basic animations, without showing how these algorithms solve real-world problems like routing or lock synchronization.

1.3 Objectives

The primary objectives of this project are:

• To engineer a transactional movie ticket booking system with a modern, high-performance UI.

• To implement Dijkstra's and A* pathfinding algorithms to compute shortest paths and sort cinemas automatically by travel distance.

• To integrate Greedy Allocators and Backtracking Cluster finders to recommend seats based on distance-to-center and group adjacency constraints.

• To implement Mutex Locks and Counting Semaphores to synchronize concurrent requests and eliminate double bookings.

• To design a DSA Dashboard that visualizes search, sorting, graph pathfinding, heap priority, and concurrency execution steps.

1.4 Scope of the Project

The scope of CineBook covers the complete reservation flow, backend synchronization APIs, and educational visualization modules. The client application is built with a responsive dashboard, and the backend handles transactional state fallbacks. The visualizer contains an execution engine that steps through pseudocode lines and displays variable states in a JSON inspector. This project does not include payment gateway integrations or real-time map API licensing, relying instead on structured coordinate networks.

1.5 Technologies Used

• React: Used to build a responsive, single-page UI with fast state updates.

• TypeScript: Used to enforce type safety across both frontend components and shared DSA modules.

• Tailwind CSS: Used to implement a premium dark-mode theme with custom animations.

• Node.js & Express: Used to build the REST API endpoints and manage memory databases.

• Framer Motion: Used to create animations for stack operations and seating recommendation passes.

• Zustand: Used to manage state across the multi-step booking wizard.

1.6 System Features

The CineBook application provides a seamless flow from movie selection to ticket verification:

• Movie Picker: Displays titles, ratings, and genre tags with official posters.

• Date Select & Proximity Finder: Prompts date selection and runs Dijkstra/A* pathfinders to sort cinemas by travel time.

• Showtime Search: Lists showtimes and runs a binary search comparison to highlight selection indexes.

• Seat Selection & Recommender: Displays interactive seat grids and runs Greedy or Backtracking recommendations.

• Checkout simulation: A sandbox that allows users to trigger concurrent transactions using Mutex, Semaphore, or Unsynchronized modes.

• Pass Generation: Generates confirmation tickets displaying the calculated HashMap index bucket slot.

2. ALGORITHMIC DESIGN AND DSA INTEGRATION

2.1 Binary Search

Purpose: Locates a specific element in a pre-sorted array by repeatedly halving the search interval.

CineBook Application: Used in the showtime list. When a user selects a timing slot, the system runs a binary search internally to find the slot's index, reducing time complexity from O(N) to O(log N).

Working Principle: 1. Initialize:

low = 0

high = N - 1

2. Repeat while low <= high:

a. Calculate:

mid = (low + high) / 2

b. If arr[mid] == target:

Return mid

c. Else if arr[mid] < target:

low = mid + 1

d. Else:

high = mid - 1

3. If the loop terminates without finding the target:

Return -1 (Element Not Found)

Pseudocode Implementation:

```
Algorithm BinarySearch(arr, target)
Input:
    arr    → Sorted array
    target → Element to search
Output:
    Index of target if found,
    otherwise -1


Begin
    low ← 0
    high ← length(arr) - 1
    while low ≤ high do
        mid ← floor((low + high) / 2)
        if arr[mid] = target then
            return mid
        else if arr[mid] < target then
            low ← mid + 1
        else
            high ← mid - 1
    end while
    return -1
End
```

Time Complexity: O(log N)

Space Complexity:O(1)

2.2 Graph Representation

Purpose: Models relationships between entities using vertices (nodes) and edges (connections) with weights.

CineBook Application: Represents the local cinema network, where vertices represent locations (User Location, PVR 4DX, INOX, etc.) and edges represent travel routes with weights representing travel distance.

Working Principle: 1. Initialize vertex list and edge adjacency lists.
2.Add vertices with coordinate properties (X,Y).
3. Add bidirectional edges with travel distance weights.

Pseudocode Implementation:

Algorithm Graph

Data:

vertices ← Empty Dictionary

Procedure AddVertex(id, coordinates)

vertices[id] ←

{

coordinates : coordinates,

edges : Empty List

}

End Procedure

Procedure AddEdge(u, v, weight)

vertices[u].edges.append(

{

```
    node   : v,
    weight : weight
  })
  vertices[v].edges.append(
  {
    node   : u,
    weight : weight
  })
```

End Procedure

Time Complexity: O(V + E) Space

Space Complexity: O(V + E)

ADVANTAGES

- Accurately represents real-world networks such as theatre locations, roads, and seat connectivity.
- Efficiently supports graph traversal and shortest-path algorithms like **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Dijkstra's Algorithm**.
- Provides flexibility to model both directed and undirected relationships.
- Simplifies route optimization and theatre recommendation in the CineBook system.
- Enables scalable representation of complex interconnected data.

Limitations

- Requires more memory than simple linear data structures, especially for dense graphs.
- Implementation is more complex due to the use of dynamic data structures such as adjacency lists or matrices.
- Traversal and pathfinding algorithms can become computationally expensive for very large graphs.

- Maintaining and updating graph relationships may increase implementation complexity.
- Performance may degrade when the graph contains a large number of vertices and edges.

2.3 Dijkstra's Algorithm

Purpose: Computes the shortest paths from a single source vertex to all other vertices in a weighted graph.

CineBook Application: Calculates the nearest travel paths from the user's location coordinates to all theaters, automatically sorting the theater cards on page load.

Working Principle: 1. Set distance[source] = 0, all other distances = Infinity.
2. Push source to Priority Queue.
3. While PQ is not empty:
   a. Dequeue node with minimum distance.
   b. For each neighbor, calculate alternative distance.
   c. If alt < distance[neighbor], update distance and push to PQ.

Pseudocode Implementation:

```
function Dijkstra(graph, start):
  distances = fill(Infinity)
  distances[start] = 0
  pq.enqueue(start, 0)
  while pq is not empty:
    curr = pq.dequeue()
    for neighbor in graph.getNeighbors(curr):
      alt = distances[curr] + neighbor.weight
      if alt < distances[neighbor]:
        distances[neighbor] = alt
        pq.enqueue(neighbor, alt)
```

Time Complexity: O((V + E) log V)

Space Complexity: O(V)

Advantages:

• Guarantees the shortest path.

• Handles arbitrary graph structures.

Limitations:

• Examines vertices blindly without directional heuristics (like A*).

• Cannot handle negative edge weights.

2.4 Queue (FIFO)

Purpose: A linear data structure that processes elements in a First-In, First-Out sequence.

CineBook Application: Manages incoming checkout requests. Concurrent transactions are placed in a FIFO buffer queue, ensuring requests are processed in order and preventing write conflicts.

Working Principle: 1. Enqueue request at the rear of the queue.

2. Dequeue request from the front for processing.

3. Check if queue is empty.

Pseudocode Implementation:

```
class Queue:
  items = []
  function enqueue(val):
    items.push(val)
  function dequeue():
    return items.shift()
```

Time Complexity: O(1) Enqueue/Dequeue

Space Complexity: O(N)

Advantages:

• Fair resource scheduling.

• Simple to implement.

Limitations:

• No random-access queries supported.

• Inefficient searches.

2.5 Priority Queue (Min Heap)

Purpose: A binary tree where each parent node is smaller than or equal to its children, allowing quick retrieval of the minimum value.

CineBook Application: Maintains the list of open seat recommendations sorted by distance to the center, retrieving the closest seat in O(1) time.

Working Principle: 1. Insert element at the end of the heap array.

2. Bubble up the element until the heap property is restored.

3. Extract the root node, swap with the last element, and bubble down.

Pseudocode Implementation:

```
class MinHeap:
  heap = []
  function insert(val):
    heap.push(val)
    bubbleUp(heap.length - 1)
  function extractMin():
    min = heap[0]
    heap[0] = heap.pop()
    bubbleDown(0)
    return min
```

Time Complexity: O(log N) Insert/Extract

Space Complexity: O(N)

Advantages:

• Constant-time retrieval of the minimum element.

• Logarithmic insertion and deletion.

Limitations:

• Slow O(N) lookup for non-minimum elements.

• Unsorted layout.

2.6 Graph Traversal (DFS/BFS)

Purpose: Traverses or searches graph structures systematically to explore connectivity.

CineBook Application: Verifies graph paths and computes layout coordinates dynamically for visual representation in the DSA Dashboard.

Working Principle: 1. Initialize queue (BFS) or stack (DFS) with start node.

2. Mark start node as visited.

3. While queue/stack is not empty:

   a. Pop node, visit it.

   b. Push unvisited neighbors to queue/stack.

Pseudocode Implementation:

```
function BFS(graph, start):
  visited = Set(), queue = [start]
  while queue is not empty:
    curr = queue.shift()
    visit(curr)
    for n in graph.getNeighbors(curr):
      if n not in visited: queue.push(n)
```

Time Complexity: O(V + E)

Space Complexity: O(V)

Advantages:

• Thoroughly scans connectivity.

• Finds path routes easily.

Limitations:

• Can consume significant memory.

• Slow search on deep structures.

2.7 Mutex & Counting Semaphore

Purpose: Synchronization primitives that enforce mutual exclusion and limit concurrent access to resources.

CineBook Application: Used in the server checkout router. Mutex locks the validation pipeline for a user to prevent double bookings, while Semaphore limits concurrent connections to 2.

Working Principle: 1. Thread calls acquire().

2. If lock is held or permits are 0, suspend thread and add to queue.

3. Else, grant access.

4. When done, call release() to notify the next thread.

Pseudocode Implementation:

```
class Mutex:
  locked = false
  queue = []
  function acquire(owner):
    if not locked:
      locked = True
    else:
      queue.push(owner)
  function release():
    if queue has items:
      next = queue.shift()
    else:
      locked = False
```

Time Complexity: O(1) Lock/Unlock

Space Complexity: O(Q) Queue size

Advantages:

• Prevents race conditions and double bookings.

• Limits thread starvation.

Limitations:

• Can cause deadlocks if not managed properly.

• Reduces throughput.

2.8 Recommendation Algorithm

Purpose: Calculates popularity scores based on rating, sales, and genres.

CineBook Application: Calculates relevance scores for movie recommendations based on user history.

Working Principle: 1. Fetch movie list.

2. Compute score = (rating * 0.6) + (popularity * 0.4).

3. Sort movies in descending order of score.

Pseudocode Implementation:

```
function RecommendMovies(movies):
  for m in movies:
    m.score = m.rating * 0.6 + m.popularity * 0.4
  sort movies by score descending
  return movies
```

Time Complexity: O(N log N) due to sorting

Space Complexity: O(N)

Advantages:

• Simple popularity metrics.

• Dynamic updating.

Limitations:

• Requires sorting.

• Basic recommendation heuristics.

2.9 Complexity Analysis Summary

The table below summarizes the computational complexities and practical applications of the key algorithms integrated into CineBook:

Algorithm	Data Structure	Time Complexity	Space Complexity	CineBook Application
Binary Search	Sorted Array	O(log N)	O(1)	Locating showtime slots
Dijkstra	Graph Adjacency List	O((V + E) log V)	O(V)	Sorting nearest theatres
A* Search	Graph Adjacency List	O(E log V) Average	O(V)	Comparing paths to theatres
Min Heap	Binary Tree Array	O(log N) Insert/Pop	O(N)	Seating priority ranks
Queue (FIFO)	Linked List Buffer	O(1) Enqueue/Dequeue	O(N)	Simulating request waits
Stack (LIFO)	Dynamic Array	O(1) Push/Pop	O(N)	Tracking selection undo
Mutex Lock	Semaphore State object	O(1)	O(Q) Wait Queue	Atomizing seat booking checks
HashMap	Chained Buckets Array	O(1) Average	O(N + B)	Gate validation lookup

3. RESULTS AND DISCUSSION

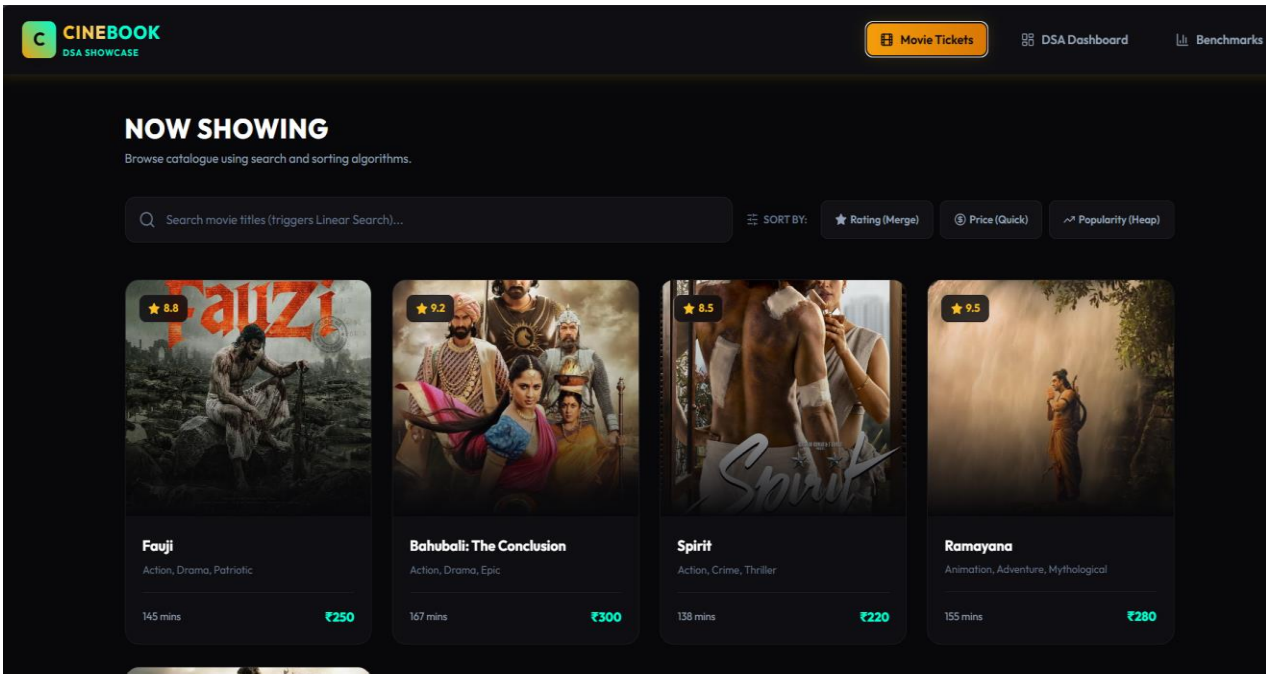
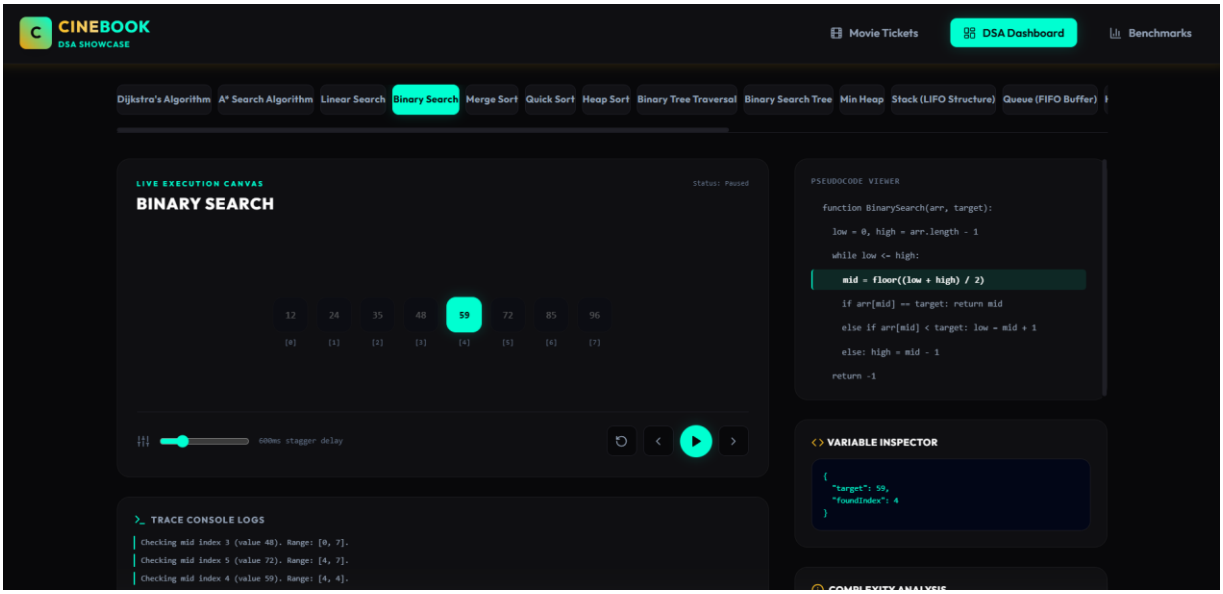


Figure 1: Movie Details

The Movie Details page displays details for the selected movie (genres, synopsis, rating) and a date selection grid, bypassing early time-slot choices.


CINEBOOK
DSA SHOWCASE
Movie Tickets
DSA Dashboard
Benchmarks
Dijkstra's Algorithm
A* Search Algorithm
Linear Search
Binary Search
Merge Sort
Quick Sort
Heap Sort
Binary Tree Traversal
Binary Search Tree
Min Heap
Stack (LIFO Structure)
Queue (FIFO Buffer)
LIVE EXECUTION CANVAS
BINARY SEARCH
Status: Paused
12
24
35
48
59
72
85
96
[0]
[1]
[2]
[3]
[4]
[5]
[6]
[7]
600ms stagger delay
PSEUDOCODE VIEWER
function BinarySearch(arr, target):
low = 0, high = arr.length - 1
while low <= high:
mid = floor((low + high) / 2)
if arr[mid] == target: return mid
else if arr[mid] < target: low = mid + 1
else: high = mid - 1
return -1
VARIABLE INSPECTOR
{
"target": 59,
"foundIndex": 4
}
TRACE CONSOLE LOGS
Checking mid index 3 (value 48). Range: [0, 7].
Checking mid index 5 (value 72). Range: [4, 7].
Checking mid index 4 (value 59). Range: [4, 4].

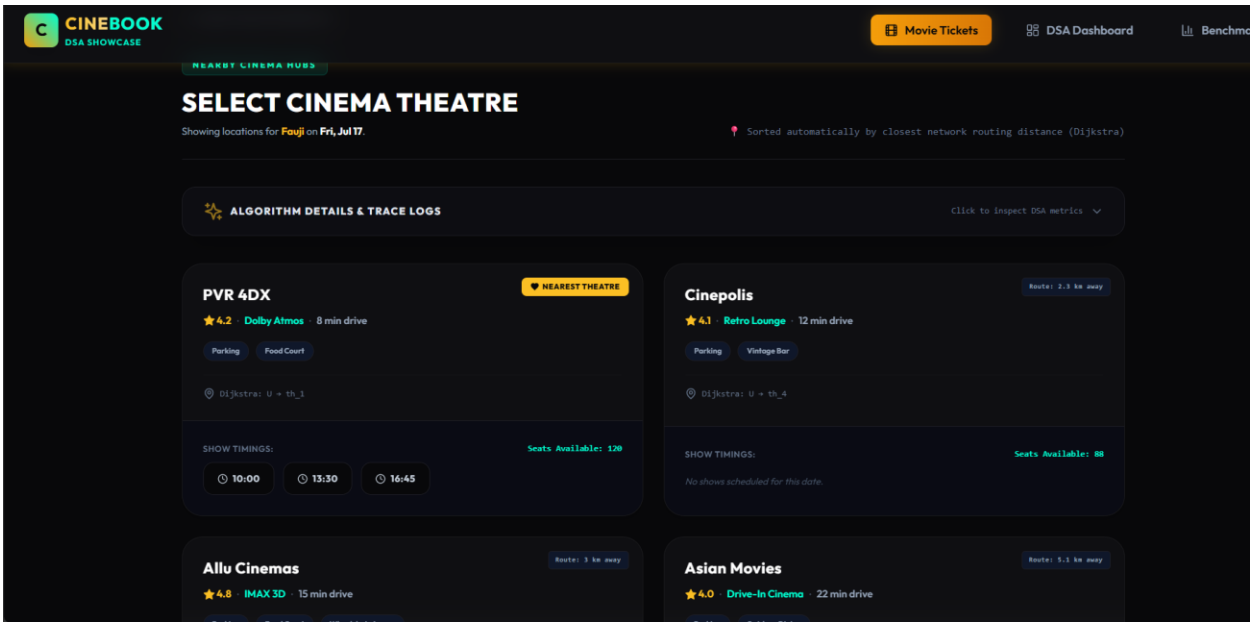


Figure 3: Theatre Selection

The Theatre Selection page lists nearby cinemas sorted automatically by travel distance computed using Dijkstra's algorithm. It displays travel time, facilities, showtimes, and an expandable algorithm trace details drawer

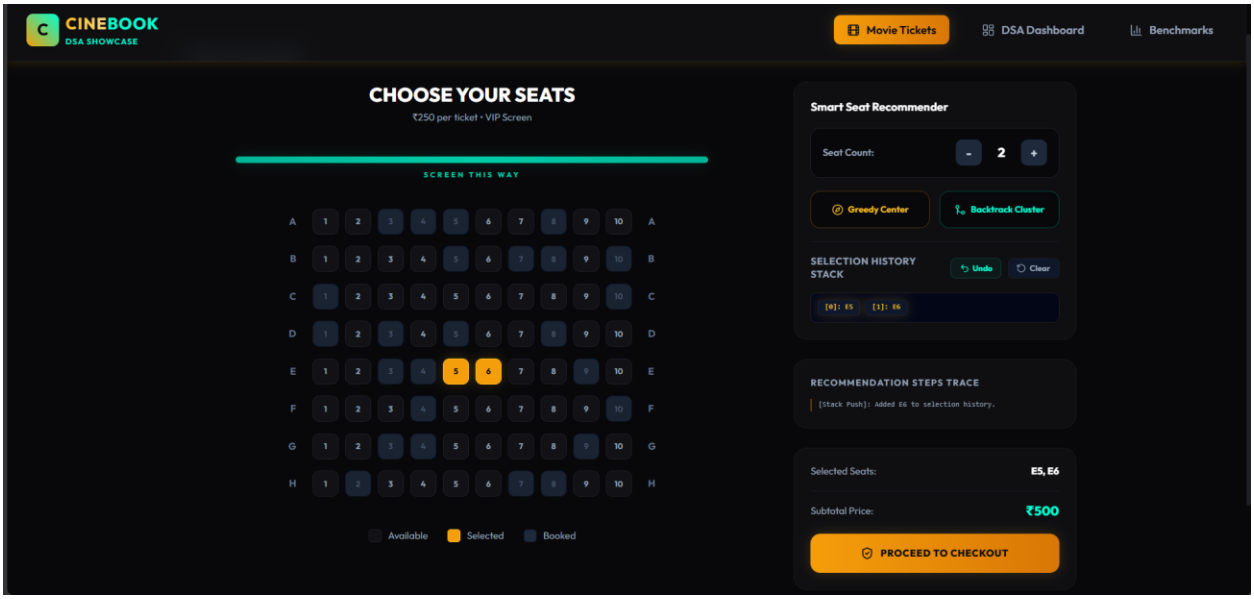


Figure 4: Seat Selection Grid

The Seat Selection page features an interactive grid layout representing the cinema hall. It displays recommended seats using Greedy or Backtracking algorithms and shows undo logs.

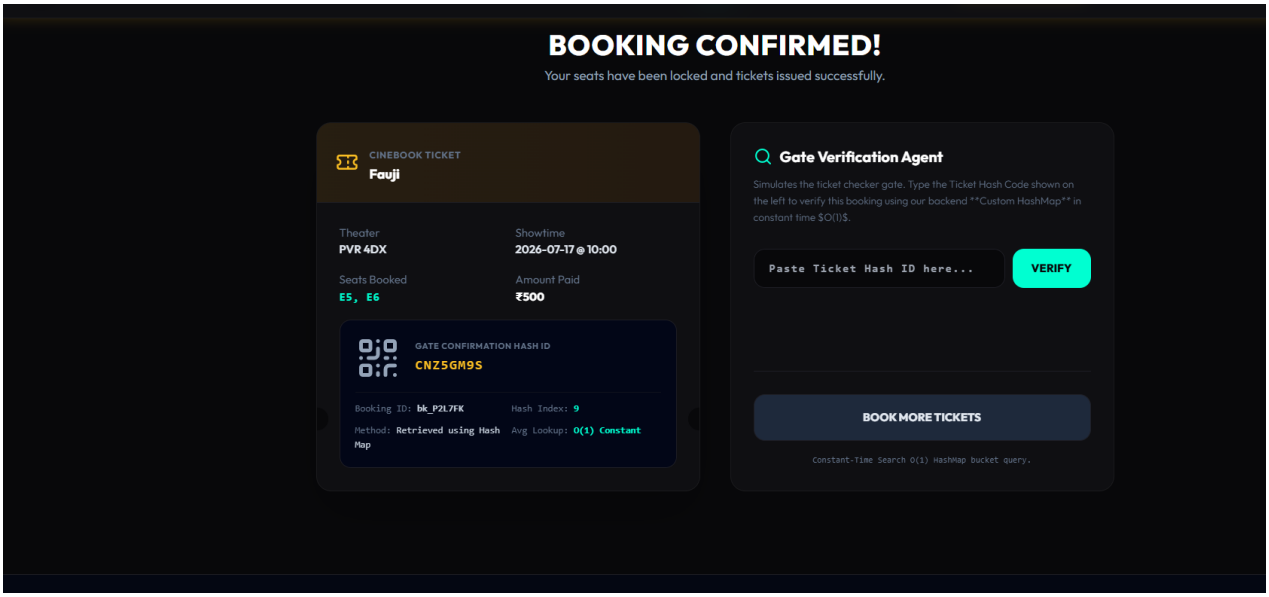


Figure 6: Booking Confirmation Ticket

The Confirmation page displays the finalized booking receipt, including a QR code and the calculated HashMap index bucket slot indicating where the ticket hash is stored in memory.

The DSA Dashboard contains step players, trace logs, pseudocode visualizers, and JSON Variable Inspectors for all algorithms, separated from the commercial booking flow.

4. CONCLUSION AND FUTURE SCOPE

4.1 Project Achievements

We successfully engineered the 'CineBook' ticket booking portal. By separating commercial workflows from the educational visualizer modules, we improved the user experience while maintaining robust theoretical algorithm traces. The Dijkstra pathfinding engine automatically sorts cinema cards, the Greedy allocator finds the best seating coordinates, and the backend Mutex locking ensures synchronization under stress, preventing double bookings.

4.2 Project Advantages

The project offers several key advantages:

• Safety: Mutex locks eliminate double bookings.

• Convenience: Pathfinding automatically sorts theaters by distance.

• Clarity: The DSA Dashboard provides visual trace logs and variable inspectors for learning.

4.3 Limitations

The current implementation runs on in-memory databases, limiting persistence. In addition, the coordinate calculations use static vertex definitions rather than dynamic maps.

4.4 Future Scope

Future enhancements could include:

• Third-Party Maps API Integration: For dynamic travel calculations.

• Persistent Database Migration: Moving from in-memory arrays to PostgreSQL/MongoDB databases.

• AI-based Seating Recommendations: Optimizing seat recommendations using user history.

4.5 Student Learning Outcomes

This project provided hands-on experience in full-stack architecture design. Key learnings include implementing synchronization primitives (Mutex, Semaphore), managing state in multi-step wizard processes using Zustand, and optimizing data layers with TypeScript.

4.6 Final Conclusion

CineBook demonstrates that fundamental Data Structures and Algorithms are not merely theoretical concepts studied in computer science curricula but powerful techniques that can be seamlessly integrated into commercial software applications to solve real-world challenges. By incorporating efficient algorithms for pathfinding, searching, seat optimization, and concurrent synchronization directly into the booking workflow, the system enhances performance, improves decision-making, and ensures reliable transaction processing under high user demand. The project illustrates how carefully selected algorithms contribute to faster execution, optimized resource utilization, improved scalability, and enhanced user satisfaction.

Furthermore, CineBook serves as a practical blueprint for developing secure, scalable, and responsive software systems capable of supporting modern online services. Its modular architecture, efficient backend synchronization mechanisms, and interactive DSA visualization dashboard demonstrate the successful combination of academic learning with industry-oriented software engineering practices. Beyond functioning as a premium movie ticket booking platform, the application also acts as an educational tool that bridges the gap between theoretical algorithm concepts and their practical implementation. This approach highlights the importance of algorithm-driven software design and provides a strong foundation for future enhancements such as AI-powered recommendations, cloud-native deployment, distributed microservices, and large-scale enterprise applications.

5. REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA: MIT Press, 2009.

[2] R. Sedgewick and K. Wayne, Algorithms, 4th ed. Upper Saddle River, NJ: Addison-Wesley, 2011.

[3] Facebook Open Source, 'React - A JavaScript library for building user interfaces,' 2026. [Online]. Available: https://react.dev.

[4] Microsoft Corporation, 'TypeScript - JavaScript with syntax for types,' 2026. [Online]. Available: https://www.typescriptlang.org.

[5] Node.js Foundation, 'Node.js - Runtime Environment,' 2026. [Online]. Available: https://nodejs.org.

[6] ExpressJS, 'Express - Fast, unopinionated, minimalist web framework for Node.js,' 2026. [Online]. Available: https://expressjs.com.

[7] Framer Foundation, 'Framer Motion - A production-ready motion library for React,' 2026. [Online]. Available: https://framer.com/motion.

[8] E. W. Dijkstra, 'A note on two problems in connexion with graphs,' Numerische Mathematik, vol. 1, pp. 269-271, 1959.

[9] P. E. Hart, N. J. Nilsson, and B. Raphael, 'A Formal Basis for the Heuristic Determination of Minimum Cost Paths,' IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100-107, 1968.

[10] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ: Wiley, 2018.